Research prototype · seeking pre-seed

The compiler that learns.

An AI that learns to optimize machine code — by understanding how data actually flows through it, not by reading text. Built on JEPA, a self-supervised world-model approach. To our knowledge, the first applied to assembly.

THE STAKES

The hardest code on earth runs on the thinnest margins.

Trading engines. Physics simulations. Databases. There, **every cycle of speed and every byte of memory matters** — and one thing is never negotiable: the code must be **correct, right down to the metal**.

ns
latency that moves money

100%
correctness, non-negotiable

$B
spent chasing performance

THE HIDDEN TRUTH

Performance isn't decided in your source code. It's decided in the assembly the compiler produces.

So why not just let the compiler handle it? That's exactly the problem.

THE PROBLEM · TODAY'S COMPILERS

Frozen heuristics that never learn.

- **Hand-tuned rules** an expert wrote years ago, applied in a **fixed order**.
- **Crude cost models** that approximate the hardware — and drift further from reality with every new chip generation.

> **We measured it:** on a real hot loop, the compiler leaves **up to 4.4×** on the table — not for lack of capability, but to stay safe. **That gap is the opportunity.**

- They must guarantee correctness, so they **play it safe** and leave performance on the table.
- They **don't learn**. Every new architecture = more manual tuning, by hand, by experts. The compiler never gets smarter on its own.

OUR INSIGHT

Represent code differently. Learn from it differently.

The structure, not the text. We turn each block of assembly into a **graph of data flow** — what depends on what. That is the real structure that drives performance. Unlike an LLM, we never act on text.

A bet on how it learns. On top of that graph we use **JEPA** — Yann LeCun's self-supervised world-model approach, just beginning to be explored in new domains (e.g. biology). **To our knowledge, no one has applied it to assembly. We're the first.**

HOW IT WORKS

From instructions to a learned world model of code.

Assembly / LLVM IR
the real program

→

Data-flow graph
control · data · call edges (ProgramML)

→

JEPA encoder
GNN, self-supervised, no labels

→

Factored embedding
$z = [\, z_{sem} \mid z_{speed} \,]$

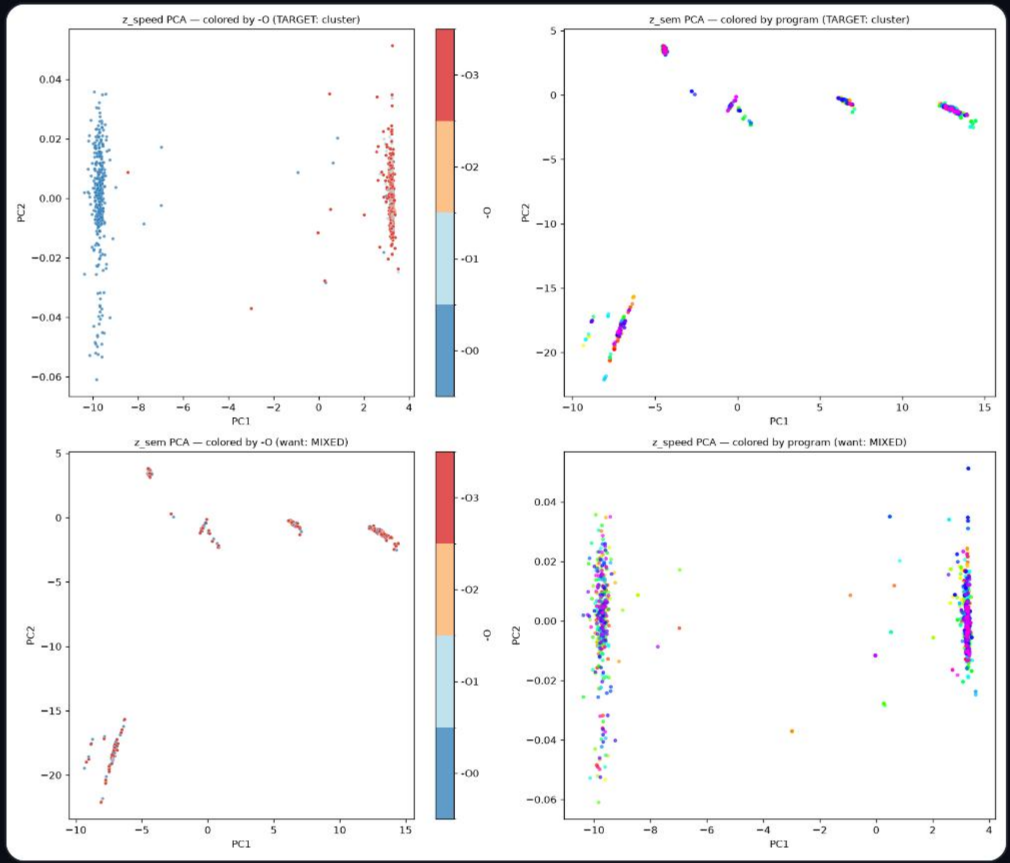
> The model splits its understanding in two: **what the code does** (z_{sem}) and **its optimization profile** (z_{speed}) — learned with zero manual labels.

PROOF · IT WORKS

The model disentangles meaning from speed.

z_{sem} = what the code does	**gap 0.89**
…ignores optimization level	**silhouette ≈ 0 ✓**
z_{speed} = optimization profile	**isolated axis**
…ignores the program	**silhouette –0.91 ✓**

Self-supervised, no labels · ~8k ExeBench functions · ProgramML graphs · trained from scratch on one B200 GPU.



z_{sem} clusters by program; z_{speed} by optimization. The two are decorrelated.

PROOF · THE TEAM DOES REAL SCIENCE

We measure honestly, and we debug deep.

- **We gate before we train.** A probe honestly quantified where today's compiler saturates (O0→O1 changes 100% of graphs; O2≈O3 are identical). We report the ceiling instead of hiding it.
- **We push the representation hard.** Careful tuning of the objective took the model from using a handful of dimensions to most of them — on the same data.

3	72
latent dims used (initial)	after tuning · same data

Effective rank of z_{sem}, 3 → 72 of 96. The kind of rigor that machine-level correctness demands.

GO-TO-MARKET · THE "GITGUARDIAN" PLAYBOOK

We don't sell promises. We send proof.

Scan
open-source GitHub repos

→

Flag
code segments we can radically optimize

→

Reach out
automated, to the repo owner

→

Free demo
on their own code

> Immediate, irrefutable proof of value — converting open source into qualified inbound, the same viral motion that built GitGuardian.

BUSINESS MODEL · WHO PAYS

We take a cut of the compute we save.

Who

Teams whose compute bill is the business.

HFT & trading, databases, simulation / HPC, ML infra, embedded — already paying perf engineers $200k+ and millions a year in compute.

Pricing

Value-based: a share of the savings.

Land via the free demo → take a % of the wall-clock / compute reduction we prove. A per-seat tier for the dev-tool motion.

Margin

Optimize once, value compounds.

Optimizing a hot loop costs cents of GPU; the speedup it buys recurs every run, forever. ~90%+ gross margin.

> The math: 5% off a **$2M/yr** compute bill = $100k saved → we keep 25% = **$25k/yr per workload**, for a few dollars of compute to produce it.

THE VISION

A universal, AI-driven compiler.

The encoder is the foundational brick. The end state: take **any source, in any language**, and compile it **optimally for any hardware** — CPU, GPU, specialized chips. A world model that gets smarter with every program it sees.

WHY NOW

The window just opened.

JEPA

self-supervised world models are crossing into new sciences — untouched in assembly

chips

each new architecture makes hand-tuning compilers more unsustainable

compute

distributed GPU markets make training economical for a small team

EXECUTION · HOW WE RUN LEAN

Massive compute, fraction of the cost.

- **Infrastructure:** distributed cloud (Vast.ai) for massive GPU power at a fraction of hyperscaler cost. Today: a single B200 trains the encoder end-to-end.
- **Data & curriculum:** training structured in **educational echelons** — the model learns algorithmics from the basics up to the most complex architectures.

ProgramML graphs | ExeBench corpus | JEPA / VICReg | PyTorch Geometric | Vast.ai · B200

Full pipeline (probe → cache → train → eval) reproduces from one command line.

WHAT YOU'RE PROBABLY ASKING · 1 / 2

The hard questions, answered honestly.

- **"Does this actually make code faster yet?"**
  Not yet — we've proven the model **separates meaning from optimization**.
  Next milestone: a first measured speedup on a real benchmark.

- **"How do you guarantee correctness?"**
  We **propose, the toolchain proves** — every AI rewrite is gated by the compiler's equivalence checks. We never ship an unverified transform.

WHAT YOU'RE PROBABLY ASKING · 2 / 2

Defensibility.

- **"Why won't LLVM / Google / an LLM lab crush you?"**
  We learn on the **data-flow graph, not text**. A JEPA world model + the optimizer loop is the method; the durable moat is a **proprietary corpus of (graph → measured speedup) pairs** — not a weekend clone.

> The moat compounds: every optimization we verify becomes a hardware-measured record no incumbent has.

Let's make the compiler learn.

Seeking pre-seed to take the encoder from **proven disentanglement** to a **first measured speedup** on real code.

‹ contact · team · the ask ›